Chapter 4: Syntax Analysis

The Art of Parsing

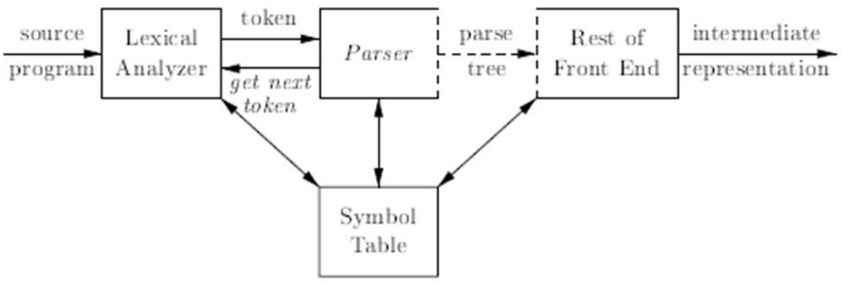
By
Dr. Adnan M. Sherif, PhD

CS432 – Spring 2025/2026

1

Introduction

- The parser (Syntax Analyser) obtains a string of tokens from the lexical analyser, and verifies that the string of token names can be generated by the grammar for the source language.



2

Expected output of a Parser

- Parse Tree:
  - The parser constructs a parse tree and passes it to the rest of the compiler for further processing.
  - The subsequent phases of the compiler (semantic analysis, code generation) walk this tree to understand the program and generate code.
  - The parse Tree is generated following the language grammar structure.
- Update Symbol Table:
  - The parser amends the Symbol Table with details obtained from the syntax of the language, (ex identifiers are classified as variables or function names.)
- Error reporting:
  - The list of errors that show points in the source program that do not follow the language grammar. "Syntax Error" is more helpful than "Invalid Character." The parser can often pinpoint where the structure breaks down.

3

Error handling

- Common programming errors can occur at many levels.
  - **Lexical errors** include misspellings of identifiers, keywords, or operators (e.g., the use of an identifier ellipse$ize instead of ellipseSize, and missing quotes around text intended as a string)
  - **Syntactic errors** include misplaced semicolons or extra or missing braces; that is, “{“ or “}." As another example, in C or Java, the appearance of a case statement without an enclosing switch is a syntactic error.
  - **Semantic errors** include type mismatches between operators and operands, e.g., the return of a value in a Java method with result type void.
  - **Logical errors** can be anything from incorrect reasoning on the part of the programmer to the use in a C program of the assignment operator = instead of the comparison operator ==. The program containing = may be well-formed; however, it may not reflect the programmer's intent.

4

Error-Recovery Strategies

- Panic-Mode Recovery
  With this method, on discovering an error, the parser discards input symbols one at a time until one of a designated set of synchronizing tokens is found. The synchronizing tokens are usually delimiters, such as semicolon or }.
- Phrase-Level Recovery
  On discovering an error, a parser may perform local correction on the remaining input; that is, it may replace a prefix of the remaining input by some string that allows the parser to continue.
- Error Productions
  By anticipating common errors that might be encountered, we can augment the grammar for the language at hand with productions that generate the erroneous constructs.
- Global Correction
  Ideally, we would like a compiler to make as few changes as possible in processing an incorrect input string. There are algorithms for choosing a minimal sequence of changes to obtain a globally least-cost correction. Given an incorrect input string x and grammar G, these algorithms will find a parse tree for a related string y, such that the number of insertions, deletions, and changes of tokens required to transform x into y is as small as possible.

5

Context Free Grammar (CFG)

- Formal definition:
  - A grammar G is defined as a quadruple
    $G = (V, \Sigma, P, S)$

  such that:

  V is the set of none terminals of the grammar
  e.g., expr, statement .. etc.

  Σ is the set of terminal symbols (tokens)
  e.g., id, + , while .. etc.

  S the stating symbol a member of V

  P production rules used to define the language.

$$
\begin{array}{rcl}
expression & \rightarrow & expression + term \\
expression & \rightarrow & expression - term \\
expression & \rightarrow & term \\
term & \rightarrow & term * factor \\
term & \rightarrow & term / factor \\
term & \rightarrow & factor \\
factor & \rightarrow & (expression) \\
factor & \rightarrow & \mathbf{id}
\end{array}
$$

6

Writing grammar

- Eliminate ambiguity

  ambiguous grammar can generate more than one parse tree for the same input sequence of tokens. We need to eliminate the ambiguity from the grammar.

  Example the dangling else problem, restructuring the grammar is needed following some rule of precedence.

$$
\begin{array}{lcl}
stmt & \rightarrow & \textbf{if } expr \textbf{ then } stmt \\
 & | & \textbf{if } expr \textbf{ then } stmt \textbf{ else } stmt \\
 & | & \textbf{other}
\end{array}
$$

if E1 then if E2 then S1 else S2

$$
\begin{array}{lcl}
stmt & \rightarrow & matched_stmt \\
 & | & open_stmt \\
matched_stmt & \rightarrow & \textbf{if } expr \textbf{ then } matched_stmt \textbf{ else } matched_stmt \\
 & | & \textbf{other} \\
open_stmt & \rightarrow & \textbf{if } expr \textbf{ then } stmt \\
 & | & \textbf{if } expr \textbf{ then } matched_stmt \textbf{ else } open_stmt
\end{array}
$$

7

Writing grammar

- Elimination of Left Recursion

  A grammar is left recursive if it has a nonterminal A such that there is a derivation $A \Rightarrow^{+} A\alpha$ for some string α.

  Some parser algorithms can not deal with left recursion.

  General rule for eliminating left recursion, for any production of the form $A \rightarrow A\alpha \mid \beta$ can be rewritten as:

$$A \rightarrow \beta A'$$
$$A' \rightarrow \alpha A' \mid \varepsilon$$

$$
\begin{array}{lcl}
stmt & \rightarrow & \textbf{declare id } optionList \\
optionList & \rightarrow & optionList\ option \mid \epsilon \\
option & \rightarrow & mode \mid scale \mid precision \mid base \\
mode & \rightarrow & \textbf{real} \mid \textbf{complex} \\
scale & \rightarrow & \textbf{fixed} \mid \textbf{floating} \\
precision & \rightarrow & \textbf{single} \mid \textbf{double} \\
base & \rightarrow & \textbf{binary} \mid \textbf{decimal}
\end{array}
$$

8

Writing grammar

- Elimination of Left Recursion

  We shown a method for elimination of direct left recursion, consider the following grammar:

  $$S \rightarrow A\ a \mid b$$
  $$A \rightarrow A\ c \mid S\ d \mid \varepsilon$$

  Note that S is left recursive when the rule $S \rightarrow A\ a$ is used followed by the rule $A \rightarrow S\ d$.

  $$S \Rightarrow Aa \Rightarrow Sda$$

  When applying the steps of the algorithm for elimination of left recursion we obtain the following grammar

  $$S \rightarrow A\ a \mid b$$
  $$A \rightarrow bdA' \mid A'$$
  $$A' \rightarrow cA' \mid adA' \mid \varepsilon$$

Algorithm for elimination of Left recursion

1) arrange the nonterminals in some order $A_1, A_2, \ldots, A_n$.
2) **for** (each i from 1 to n) {
3) **for** (each j from 1 to $i-1$) {
4) replace each production of the form $A_i \rightarrow A_j\gamma$ by the productions $A_i \rightarrow \delta_1\gamma \mid \delta_2\gamma \mid \cdots \mid \delta_k\gamma$, where $A_j \rightarrow \delta_1 \mid \delta_2 \mid \cdots \mid \delta_k$ are all current A_j-productions
5) }
6) eliminate the immediate left recursion among the A_i-productions
7) }

9

Writing grammar

- Left Factoring

  Left factoring is a grammar transformation that is useful for producing a grammar suitable for predictive, or top-down, parsing.

  When the choice between two alternative A-productions is not clear; we may be able to rewrite the productions to defer the decision until enough of the input has been seen that we can make the right choice.

  Example:

  $$stmt \rightarrow if\ (expr) then\ stmt\ else\ stmt$$
  $$\mid if\ (expr) then\ stmt$$

  can be left factors as follows:

  $$stmt \rightarrow if\ (expr) then\ stmt\ elsestmt$$
  $$elsestmt \rightarrow else\ stmt \mid \varepsilon$$

10

Non-Context-Free Language Constructs

- A few syntactic constructs found in typical programming languages cannot be specified using grammars alone.
- Examples:
  - The condition that a variable must be declared before it can be used.
  - The number of parameters in a function call must meet the declaration of the function.
- The above problems are solved by the next phase of the compiler (Semantic Analysis).

11

Types of Parsers

- Two Philosophies of Parsing
  - Top-Down: Start at the root (start symbol) and apply productions to build the tree down to the leaves. "Which production should I apply?"
  - Bottom-Up: Start at the leaves (tokens) and apply productions in reverse to build the tree up to the root. "Which production can I reduce?"

12

Recursive Descent Parsing (Top-down parser)

- A recursive-descent parsing program consists of a set of procedures, one for each nonterminal. Execution begins with the procedure for the start symbol, which halts and announces success if its procedure body scans the entire input string.

```
void A() /* a is the current input symbol */
1)   Choose an A production A → X1X2 ··· Xk;
2)   for ( i= 1 to k )
3)     if ( Xi is a nonterminal )
4)       call procedure Xi();
5)     else if ( Xi equals the current input symbol a )
6)           advance the input to the next symbol;
7)     else /* an error has occurred */;
```

Problems:
1) Nondeterministic, in step 1 the algorithm choice of production to use is aleatory.
2) The final else can only indicate an error if all productions for A are exhausted.
This requires some mechanism of back tracking.
3) No left recursion allowed in grammar. Why??

13

Recursive Descent Parsing

- The construction of both top-down and bottom-up parsers is aided by two functions, FIRST and FOLLOW, associated with a grammar G.
- During top-down parsing, FIRST and FOLLOW allow us to choose which production to apply, based on the next input symbol. During panic-mode error recovery, sets of tokens produced by FOLLOW can be used as synchronizing tokens.

14

First and Follow

- Define $FIRST(\alpha)$, were α is any string of grammar symbols, to be the set of terminals that begin strings derived from α. if $\alpha \overset{*}{\Rightarrow} \varepsilon$ then ε is also in $FIRST(\alpha)$.
- To compute $FIRST(X)$ for all grammar symbols X, apply the following rules until no more terminals or ε can be added:
  - If X is a terminal, then $FIRST(X) = \{X\}$
  - if X is a nonterminal with production $X \rightarrow Y_1 Y_2 \ldots Y_k$ then:
    - for i=1 to k
      - $FIRST(X) = FIRST(X) \cup (FIRST(Y_i) - \{\varepsilon\})$
      - if $\varepsilon \notin FIRST(Y_i)$ then break
    - if all $FIRST(Y_i)$ contain ε then add ε to $FIRST(X)$

15

First and Follow

- Define $FOLLOW(A)$, for nonterminal A, to be the set of terminals a that can appear immediately to the right of A in some sentential form.
- To compute $FOLLOW(A)$ for all nonterminal A, apply the following rules until nothing can be added to any $FOLLOW$ set.
  - Place \$ in $FOLLOW(S)$, where S is the start symbol, and \$ is the input right end marker (indicates the end of input).
  - If there is a production $A \rightarrow \gamma B \beta$, then everything in $FIRST(\beta)$ except ε is in $FOLLOW(B)$.
  - If there is a production $A \rightarrow \gamma B$, or $A \rightarrow \gamma B \beta$ and ε in $FIRST(\beta)$, then everything in $FOLLOW(A)$ is in $FOLLOW(B)$.

16

First and Follow

- Example: Consider the following grammar

$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \varepsilon$$
$$F \rightarrow (E) \mid id$$

Find the First and Follow of each term in the grammar.

$First(E) = First(T) = First(F) = \{(, id\}$
$First(E') = \{+, \varepsilon\}$
$First(T') = \{*, \varepsilon\}$

$Follow(E) = Follow(E') = \{\$,)\}$
$Follow(T) = Follow(T') = \{+, \$,)\}$
$Follow(F) = \{*, +, \$,)\}$

17

LL(1) Grammars

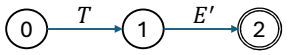
- Predictive parsers, that is, recursive-descent parsers needing no backtracking, can be constructed for a class of grammars called LL(1).
- The first "L" in LL(1) stands for scanning the input from left to right, the second "L" for producing a leftmost derivation, and the "1" for using one input symbol of lookahead at each step to make parsing action decisions.
- The class of LL(1) grammars is rich enough to cover most programming constructs, although care is needed in writing a suitable grammar for the source language. For example, no left-recursive or ambiguous grammar can be LL(1).

18

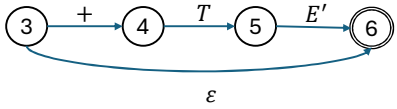
Predictive Parsers

- Transition Diagrams for Predictive Parsers
  - Transition diagrams are useful for visualizing predictive parsers.
  - To construct the transition diagram from a grammar, first eliminate left recursion and then left factor the grammar. Then, for each nonterminal A,
    - Create an initial and final (return) state.
    - For each production $A \rightarrow X_1X_2X_3 \ldots X_k$ create a path from the initial to the final state, with edges labelled $X_1X_2X_3 \ldots X_k$. If $A \rightarrow \varepsilon$, the path is an edge labelled ε .

$E \rightarrow TE'$



$E' \rightarrow +TE' \mid \varepsilon$



19

Predictive Parsers

- Predictive parsers can be constructed for LL(1) grammars since the proper production to apply for a nonterminal can be selected by looking only at the current input symbol.
- A predictive parsing table M[A; a], a two-dimensional array, where A is a nonterminal, and a is a terminal or the symbol $, the input end marker. The elements of the array are the production to be used by the parser.
- A predictive parsing table is constructed using the FIRST and FOLLOW of the production in the grammar.

20

Predictive Parsers

- Construction of a predictive parsing table.
  - The following algorithm describes the method for creating a predictive parsing table:
  - For each production $A \rightarrow \alpha$ of the grammar, do the following:
    1. For each terminal **a** in $FIRST(\alpha)$, add $A \rightarrow \alpha$ to M[A; a].
    2. If **ε** is in $FIRST(\alpha)$, then for each terminal **b** in $FOLLOW(A)$, add $A \rightarrow \alpha$ to M[A; b]. If **ε** is in $FIRST(\alpha)$ and **$** is in $FOLLOW(A)$, add $A \rightarrow \alpha$ to M[A; $] as well.

21

Predictive Parsers

- Example: Construct the predictive parsing table for the following grammar

$$E \rightarrow TE'$$
$$E' \rightarrow +TE' \mid \varepsilon$$
$$T \rightarrow FT'$$
$$T' \rightarrow *FT' \mid \varepsilon$$
$$F \rightarrow (E) \mid id$$

Non-Terminal	id	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$		
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$			$T \rightarrow FT'$		
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$			$F \rightarrow (E)$		

$First(E) = First(T) = First(F) = \{(, id\}$
$First(E') = \{+, \varepsilon\}$
$First(T') = \{*, \varepsilon\}$

$Follow(E) = Follow(E') = \{\$,)\}$
$Follow(T) = Follow(T') = \{+, \$,)\}$
$Follow(F) = \{*, +, \$,)\}$

22

Predictive Recursive Descent Parsing

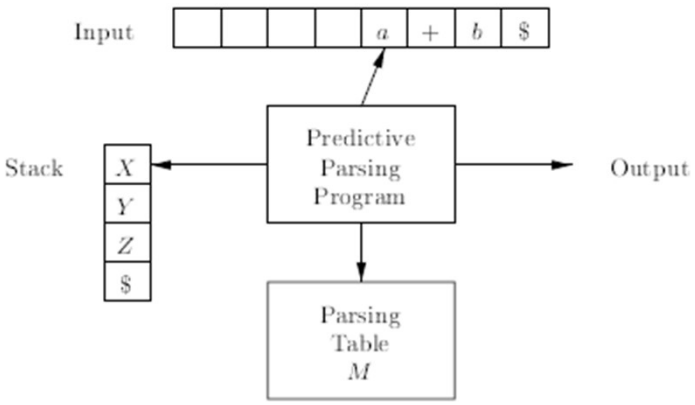
- To eliminate the problem of aleatory choice of production to use, we can change the algorithm to use the predictive parser table. The following is an emended version of the previous algorithm.

```
void A()    /* a is the current input symbol */
1)   Select production in cell M[A,a]  to be A ➔ X1X2 ··· Xk
2)   if no production found an error occurred (Error recovery).
3)   for ( i= 1 to k )
3)       if ( Xi is a nonterminal )
4)           call procedure Xi();
5)       else if ( Xi equals the current input symbol a )
6)                advance the input to the next symbol;
7)       else /* an error has occurred */;
```

23

Non-recursive Predictive Parsing

- A non-recursive predictive parser can be built by maintaining a stack explicitly, rather than implicitly via recursive calls.



24

Non-recursive Predictive Parser

- The following is the algorithm for Non-recursive Predictive Parser

let a be the first symbol of w;
let X be the top stack symbol;
while ($X \neq \$$) { /* stack is not empty */
    if ($X = a$) pop the stack and let a be the next symbol of w;
    else if (X is a terminal) error();
    else if (M[X; a] is an error entry) error();
    else if (M[X; a] = $X \rightarrow Y_1 Y_2 \ldots Y_k$){
        output the production $X \rightarrow Y_1 Y_2 \ldots Y_k$; /* some processing */
        pop the stack;
        push $Y_k Y_{k+1} \ldots Y_1$ onto the stack, with Y_1 on top; }
    let X be the top stack symbol;
}

25

Non-recursive Predictive Parser

- Class Exercise: Use the grammar and parser table from the previous example to show the parser steps to parse the following:

$$a + (b * c)\$$$

Consider a, b and c as id tokens.

26

Error Recovery in Predictive Parsing

- An error is detected during predictive parsing when the terminal on top of the stack does not match the next input symbol or when nonterminal A is on top of the stack, a is the next input symbol, and M[A; a] is error (i.e., the parsing-table entry is empty).
- Panic Mode Recovery: Panic-mode error recovery is based on the idea of skipping over symbols on the input until a token in a selected set of synchronizing tokens appears.
- As a starting point, place all symbols in FOLLOW(A) into the synchronizing set for nonterminal A. This can be achieved by adding a new entry "Synch" in the parser table.
- The recovery process is to skip tokens until an element of FOLLOW(A) is seen and pop A from the stack, it is likely that parsing can continue. if an element of FIRST(A) is found is subsequent tokens first then proceed with parsing.

27

Error Recovery in Predictive Parsing

- Class Exercise: Show how the behavior of the previous exercise predictive parser when the input is: $)a +* b\$$

$E \rightarrow TE'$
$E' \rightarrow +TE' \mid \varepsilon$
$T \rightarrow FT'$
$T' \rightarrow *FT' \mid \varepsilon$
$F \rightarrow (E) \mid id$

Non-Terminal	id	+	*	(	)	$
E	$E \rightarrow TE'$			$E \rightarrow TE'$	Synch	Synch
E'		$E' \rightarrow +TE'$			$E' \rightarrow \varepsilon$	$E' \rightarrow \varepsilon$
T	$T \rightarrow FT'$	Synch		$T \rightarrow FT'$	Synch	Synch
T'		$T' \rightarrow \varepsilon$	$T' \rightarrow *FT'$		$T' \rightarrow \varepsilon$	$T' \rightarrow \varepsilon$
F	$F \rightarrow id$	Synch	Synch	$F \rightarrow (E)$	Synch	Synch

$First(E') = \{+, \varepsilon\}$
$First(T') = \{*, \varepsilon\}$
$Follow(F) = \{*, +, \$,)\}$
$Follow(T) = Follow(T') = \{+, \$,)\}$
$First(E) = First(T) = First(F) = \{(, id\}$ $Follow(E) = Follow(E') = \{\$,)\}$

28

Bottom-Up Parsing

- A bottom-up parse corresponds to the construction of a parse tree for an input string beginning at the leaves (the bottom) and working up towards the root (the top).
- We can think of bottom-up parsing as the process of "reducing" a string w to the start symbol of the grammar. At each reduction step, a specific substring matching the body of a production is replaced by the nonterminal at the head of that production.
- The key decisions during bottom-up parsing are about when to reduce and about what production to apply, as the parse proceeds.

29

Handle Pruning

- Informally, a "handle" is a substring that matches the body of a production, and whose reduction represents one step along the reverse of a rightmost derivation.
- Example: For the following expression grammar, we have the following handle table.

$$E \rightarrow E + T | T$$
$$T \rightarrow T * F | F$$
$$F \rightarrow (E) | id$$

Consider the following input string $id * id$

$id * id$: Handle for id is used, id is reduced;
$F * id$: Handle for F is used, F is reduced;
$T * id$: Handle for T is not used, and skip T;
$T * id$: No Handle for $T *$, and skip $T *$;
$T * id$: Handle for id is used , id is reduced;
$T * F$: Handle for $T * F$ is used , $T * F$ is reduced;
T : Handle for T is used , T is reduced;
E : Reached the start symbol of Grammar ;

The above reduction is the inverse of RMD
$E \Rightarrow T \Rightarrow T * F \Rightarrow F * F \Rightarrow id * F \Rightarrow id * id$

Handle	Production to use
(E)	$F \rightarrow (E)$
id	$F \rightarrow id$
F	$T \rightarrow F$
$T * F$	$T \rightarrow T * F$
T	$E \rightarrow T$
$E + T$	$E \rightarrow E + T$

30

Shift-Reduce Parsing

- Shift-reduce parsing is a form of bottom-up parsing in which a stack holds grammar symbols and an input buffer holds the rest of the string to be parsed.
- Conventionally, when discussing bottom-up parsing, we show the top of the stack on the right, rather than on the left as we did for top-down parsing.

31

Shift-Reduce Parsing

- While the primary operations are shift and reduce, there are four possible actions a shift-reduce parser can make:
  1. **Shift**. Shift the next input symbol onto the top of the stack.
  2. **Reduce**. The right end of the string to be reduced must be at the top of the stack. Locate the left end of the string within the stack and decide with what nonterminal to replace the string.
  3. **Accept**. Announce successful completion of parsing.
  4. **Error**. Discover a syntax error and call an error recovery routine.

32

Shift-Reduce Parsing

- Example: Consider the same grammar and input from the previous example.

Stack	Input	Action
$	$Id * id$$	Shift id
$$id$	$* id$$	Reduce $F \rightarrow id$
$$F$	$* id$$	Reduce $T \rightarrow F$
$$T$	$* id$$	Shift $*$
$$T *$	$id$$	Shift id
$$T * id$	$	Reduce $F \rightarrow id$
$$T * F$	$	Reduce $T \rightarrow T * F$
$$T$	$	Reduce $E \rightarrow T$
$$E$	$	*Accept*

$E \rightarrow E + T | T$
$T \rightarrow T * F | F$
$F \rightarrow (E) | id$

33

Conflicts During Shift-Reduce Parsing

- There are context-free grammars for which shift-reduce parsing cannot be used.
- Every shift-reduce parser for such a grammar can reach a configuration in which the parser, knowing the entire stack and the next k input symbols:
  - cannot decide whether to shift or to reduce (a shift/reduce conflict).
  - cannot decide which of several reductions to make (a reduce/reduce conflict).

34

Conflicts During Shift-Reduce Parsing

- A shift/reduce conflict: Consider the following ambiguous grammar with the dangling else problem:

$$
\begin{array}{rcl}
stmt & \rightarrow & \textbf{if } expr \textbf{ then } stmt \\
 & | & \textbf{if } expr \textbf{ then } stmt \textbf{ else } stmt \\
 & | & \textbf{other}
\end{array}
$$

With the following stack

STACK	INPUT
$\cdots$ **if** *expr* **then** *stmt*	**else** $\cdots$ \$

we cannot tell whether **if** *expr* **then** *stmt* is the handle, no matter what appears below it on the stack. Here there is a shift/reduce conflict.

35

Conflicts During Shift-Reduce Parsing

- Reduce/Reduce Conflict: Consider the following grammar

$$
\begin{array}{rcl}
stmt & \rightarrow & \textbf{id (} parameter_list \textbf{)} \\
stmt & \rightarrow & expr \texttt{ := } expr \\
parameter_list & \rightarrow & parameter_list \texttt{ , } parameter \\
parameter_list & \rightarrow & parameter \\
parameter & \rightarrow & \textbf{id} \\
expr & \rightarrow & \textbf{id (} expr_list \textbf{)} \\
expr & \rightarrow & \textbf{id} \\
expr_list & \rightarrow & expr_list \texttt{ , } expr \\
expr_list & \rightarrow & expr
\end{array}
$$

36

Introduction to LR Parsing: Simple LR

- The most prevalent type of bottom-up parser today is based on a concept called LR(k) parsing; the "L" is for left-to-right scanning of the input, the "R" for constructing a right most derivation in reverse, and the k for the number of input symbols of lookahead that are used in making parsing decisions.
- we shall only consider LR parsers with k <= 1 here. When (k) is omitted, k is assumed to be 1.

37

Why LR Parsers?

- Advantages:
  - LR parsers can be constructed to recognize virtually all programming-language constructs for which context-free grammars can be written.
  - The LR-parsing method is the most general nonbacktracking shift-reduce parsing method known.
  - An LR parser can detect a syntactic error as soon as it is possible to do so, on a left-to-right scan of the input.
- Disadvantages:
  - The principal drawback of the LR method is that it is too much work to construct an LR parser by hand for a typical programming-language grammar.

38

Items and the LR(0) Automaton

- An LR parser makes shift-reduce decisions by maintaining states to keep track of where we are in a parse. States represent sets of "items".
- An LR(0) item (item for short) of a grammar G is a production of G with a dot at some position of the body. Thus, production $A \rightarrow XYZ$ yields the four items:

$$A \rightarrow .XYZ$$
$$A \rightarrow X.YZ$$
$$A \rightarrow XY.Z$$
$$A \rightarrow XYZ.$$

- The production $A \rightarrow \varepsilon$ generates only one item, $A \rightarrow$.

39

LR(0) Automaton

- One collection of sets of LR(0) items, called the canonical LR(0) collection, provides the basis for constructing a deterministic finite automaton that is used to make parsing decisions.
- Each state of the LR(0) automaton represents a set of items in the canonical LR(0) collection.
- To construct the canonical LR(0) collection for a grammar, we define an augmented grammar and two functions, CLOSURE and GOTO.

40

Augmented grammar

- If G is a grammar with start symbol S, then G', the augmented grammar for G, is G with a new start symbol S' and production $S' \rightarrow S$.
- Example:

Original Grammar

$$E \rightarrow E + T | T$$
$$T \rightarrow T * F | F$$
$$F \rightarrow (E) | id$$

Augmented grammar

$$E' \rightarrow E$$
$$E \rightarrow E + T | T$$
$$T \rightarrow T * F | F$$
$$F \rightarrow (E) | id$$

41

Closure of Item Sets

- If I is a set of items for a grammar G, then $CLOSURE(I)$ is the set of items constructed from I by the two rules:
  - Initially, add every item in I to $CLOSURE(I)$.
  - If $A \rightarrow \alpha . B\beta$ is in $CLOSURE(I)$ and $B \rightarrow \gamma$ is a production, then add the item $B \rightarrow . \gamma$ to $CLOSURE(I)$, if it is not already there. Apply this rule until no more new items can be added to $CLOSURE(I)$.
- Example: Using the previous example augmented grammar. If I is the set of one item $\{E' \rightarrow . E\}$, then $CLOSURE(I)$ is the set:

$$\{E' \rightarrow . E, E \rightarrow . E + T, E \rightarrow . T, T \rightarrow . T * F, T \rightarrow . F, F \rightarrow . id, F \rightarrow . (E)\}$$

$$E' \rightarrow E$$
$$E \rightarrow E + T | T$$
$$T \rightarrow T * F | F$$
$$F \rightarrow (E) | id$$

42

The Function GOTO

- The second useful function is $GOTO(I, X)$ where I is a set of items and X is a grammar symbol. $GOTO(I, X)$ is defined to be the closure of the set of all items $\{A \to \alpha X.\beta\}$ such that $\{A \to \alpha.X\beta\}$ is in I.
- Example: If I is the set of two items $\{E' \to E., E \to E.+T\}$, then

$$GOTO(I, +) = CLOSURE(\{E \to E+.T\}) = \begin{Bmatrix} E \to E+.T, \\ T \to .T * F, \\ T \to .F \\ F \to .(E) \\ F \to .id \end{Bmatrix}$$

43

The canonical LR(0) collection

- The algorithm to construct C, the canonical collection of sets of LR(0) items for an augmented grammar G' is:

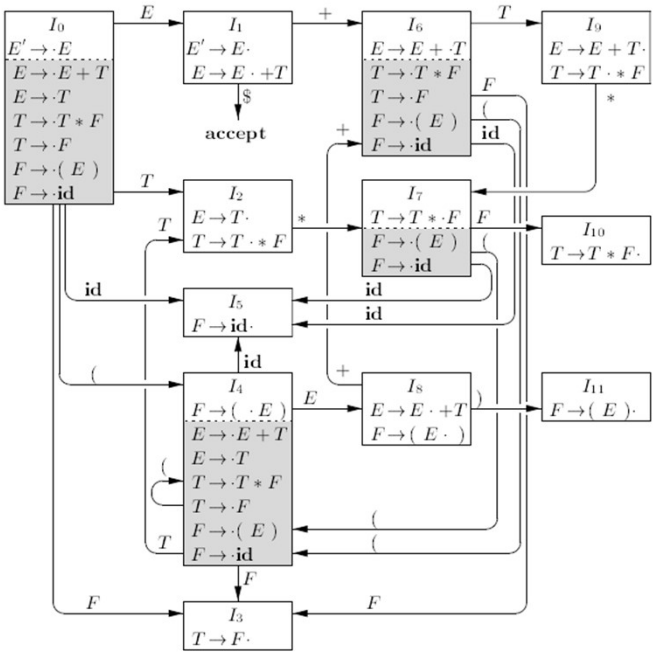
```
void items(G') {
    C = {CLOSURE({[S' → ·S]})};
    repeat
        for ( each set of items I in C )
            for ( each grammar symbol X )
                if ( GOTO(I, X) is not empty and not in C )
                    add GOTO(I, X) to C;
    until no new sets of items are added to C on a round;
}
```

44

Example

- Using the algorithm to construct the canonical set of LR parser for grammar G' in previous example.

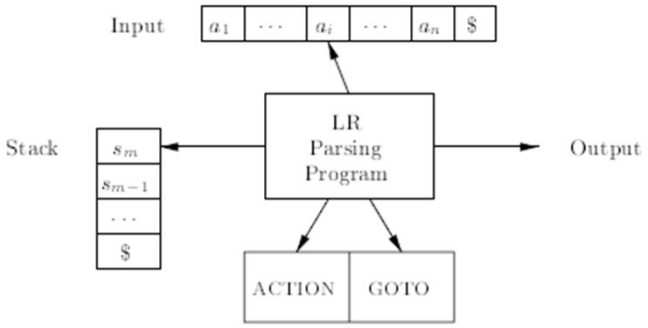
$$E' \rightarrow E$$
$$E \rightarrow E + T | T$$
$$T \rightarrow T * F | F$$
$$F \rightarrow (E) | id$$



45

The LR-Parsing Algorithm

- The LR-Parser is consistent of an input stream terminated by $, an output, a stack with $ on the top of stack at start, a driver program, and a parsing table that has two parts (ACTION and GOTO).



46

Structure of the LR Parsing Table

- The parsing table consists of two parts: a parsing-action function $ACTION$ and a go to function $GOTO$.
  - The $ACTION$ function takes as arguments a state i and a terminal a (or $ the input end marker). The value of $ACTION[i;\ a]$ can have one of four forms:
    (a) Shift j, where j is a state. The action taken by the parser effectively shifts input a to the stack, but uses state j to represent a.
    (b) Reduce $A \rightarrow \beta$. The action of the parser effectively reduces β on the top of the stack to head A.
    (c) Accept. The parser accepts the input and finishes parsing.
    (d) Error. The parser discovers an error in its input and takes some corrective action.
  - We extend the $GOTO$ function, defined on sets of items, to states: if $GOTO[Ii; A] = I_j$, then $GOTO$ also maps a state i and a nonterminal A to state j.

47

Constructing SLR-Parsing Tables

- To construct a SLR-Parsing table follow the steps:
  1. Construct $C = \{I_0, I_1, \ldots I_n\}$, the collection of sets of LR(0) items for G'.
  2. State i is constructed from I_i. The parsing actions for state i are determined as follows:
     1. If $A \rightarrow \alpha.a\beta$ is in I_i and $GOTO(I_i;\ a) = I_j$, then set $ACTION[i;\ a]$ to $shift\ j$. Here a must be a terminal.
     2. If $A \rightarrow \alpha.$ is in I_i, then set $ACTION[i;\ a]$ to $reduce\ A \rightarrow \alpha$ for all a in $FOLLOW(A)$; here A may not be S'.
     3. If $S' \rightarrow S.$ is in I_i, then set $ACTION[i;\ \$]$ to "$accept$".
  3. The goto transitions for state i are constructed for all non-terminals A using the rule: If $GOTO(I_i; A) = I_j$, then $GOTO[i; A] = j$.
  4. All entries not defined by rules (2) and (3) are made "$error$".
  5. The initial state of the parser is the one constructed from the set of items containing $S' \rightarrow .S$.

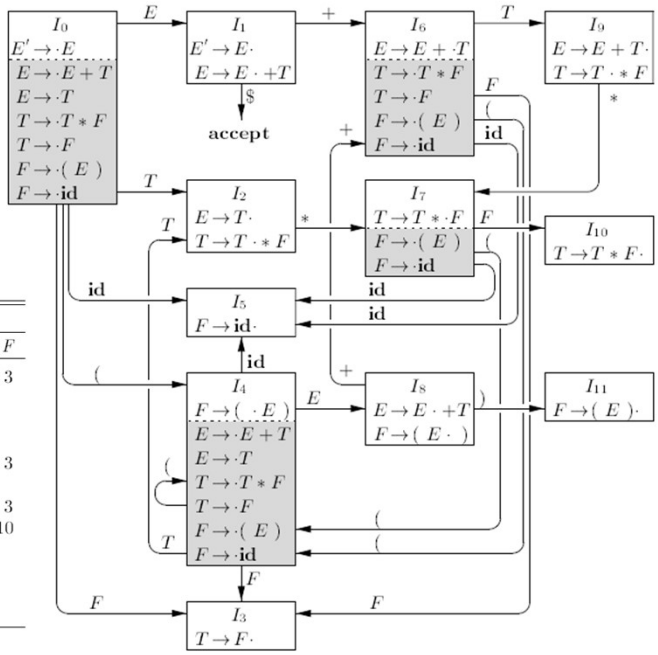
48

Example

1. $E \to E + T$
2. $E \to T$
3. $T \to T * F$
4. $T \to F$
5. $F \to (E)$
6. $F \to id$

- Using the previous example canonical set we can construct the following table:

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			



49

LR-parsing algorithm

- Initially, the parser has s_0 on its stack, where s_0 is the initial state, and $w\$$ in the input buffer. The parser then executes the program:

```
let a be the first symbol of w$;
while(1){ /* repeat forever */
    let s be the state on top of the stack;
    if ( ACTION[s, a] = shift t ) {
        push a and t onto the stack; /* in this order for state to be always on the top of the stack*/
        let a be the next input symbol;
    } else if ( ACTION[s, a] = reduce A → α ){
        pop j * 2 symbols off the stack and j = |α|;
        let state t now be on top of the stack;
        push A and GOTO[t, A] onto the stack;
        output the production A → α ;
    } else if ( ACTION[s, a] = accept ) {
        break; /* parsing is done */
    } else call error-recovery routine;
}
```

50

Example

1. $E \rightarrow E + T$
2. $E \rightarrow T$
3. $T \rightarrow T * F$
4. $T \rightarrow F$
5. $F \rightarrow (E)$
6. $F \rightarrow id$

- Use the SLR-parser table in the previous example to show parser actions to recognize the following $id + id$:

STATE	ACTION						GOTO		
	id	+	*	(	)	$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

Stack	Input	Description
$0	$id + id$	Shift 5 (add $id5$ to stack)
0id5$	$+id$	Reduce $F \rightarrow id$ (remove id5 from the stack and add $F3$ to stack)
0F3$	$+id$	Reduce T → F (remove $F3$from the stack and add to T2 to stack)
0T2$	$+id$	Reduce E → T (remove T2from the stack and add to E1to stack)
0E1$	$+id$	Shift 6 (add +6 to stack)
0E1 + 6$	id	Shift 5 (add $id5$ to stack)
0E1 + 6id5$	$	Reduce $F \rightarrow id$ (remove id5 from the stack and add $F3$ to stack)
0E1 + 6F3$	$	Reduce T → F (remove $F3$from the stack and add to T9 to stack)
0E1 + 6T9$	$	Reduce $E \rightarrow E + T$ (remove T9from the stack and add to E1 to stack)
0E1$	$	Accept

51

Error Recovery in LR Parsing

- An LR parser will detect an error when it consults the parsing action table and finds an error entry.
- In LR parsing, we can implement panic-mode error recovery as follows:
  - Scan down the stack until a state s with a goto on a particular nonterminal A is found.
  - Zero or more input symbols are then discarded until a symbol a is found that can legitimately follow A ($a \in Follow(A)$).
  - The parser then stacks the state $GOTO(s; A)$ and resumes normal parsing.

52

Error Recovery in LR Parsing

- We can improve the error messages of the parse by adding actions for error in the action table.
- Consider the previous example:

Error code	Message
e1	Error expected (‘id’ or ‘(‘) found (‘+’,’*’,’)’)
e2	Error missing operator or) mismatch
e3	Error bad formed expression expected ‘+’ or extra ‘)’
e4	Error Expression missing

STATE	ACTION						GOTO		
	id	+	*	(	)	$	*E*	*T*	*F*
0	s5	e1	e1	s4	e1	e4	1	2	3
1	e3	s6	e3	e3	e3	acc			
2	e2	r2	s7	e2	r2	r2			
3	e2	r4	r4	e2	r4	r4			
4	s5	e1	e1	s4	e1	e1	8	2	3
5	e2	r6	r6	e2	r6	r6			
6	s5	e1	e1	s4	e1	e1		9	3
7	s5	e1	e1	s4	e1	e1			10
8	e3	s6	e3	e3	s11	e3			
9	e2	r1	s7	e2	r1	r1			
10	e2	r3	r3	e2	r3	r3			
11	e2	r5	r5	e2	r5	r5			

53